

Proyecto 3

Batalla Naval: juego de dos jugadores sobre un microprocesador RISC-V con periférico VGA

Profesor

Dr.-Ing. Jeferson González-Gómez
Ing. Rolén Coto Calderón

1. Introducción

Este tercer proyecto integra, por primera vez en el curso, el diseño de un microprocesador propio basado en la arquitectura RISC-V con el desarrollo de periféricos mapeados en memoria y la generación de video mediante VGA. A diferencia de los Proyectos 1 y 2, donde el control de la aplicación se describía mediante lógica combinacional, secuencial y máquinas de estado dedicadas en SystemVerilog, en este proyecto el comportamiento de la aplicación se describe como un **programa en lenguaje ensamblador** que se ejecuta sobre un procesador diseñado por el propio equipo. El hardware que los estudiantes desarrollan deja entonces de ser la aplicación en sí misma, y pasa a ser la plataforma computacional — núcleo, memorias y periféricos — sobre la cual dicha aplicación corre.

La aplicación a desarrollar es una versión sencilla del clásico juego de *Batalla Naval* (*Battleship*) para dos jugadores. El **Jugador 1** interactúa físicamente con la tarjeta de FPGA: observa su tablero y el del oponente en el monitor VGA, y coloca barcos y dispara mediante los botones locales. El **Jugador 2** interactúa de forma remota mediante una **aplicación de PC** (por ejemplo, en Python) diseñada por el propio equipo, que se comunica con la FPGA por UART: dicha aplicación despliega en la PC el tablero del Jugador 2 y el estado conocido del tablero rival, y transmite hacia la FPGA las coordenadas de colocación de barcos y de disparo. **Todo el control de la partida — colocación, turnos, validación de disparos, detección de barcos hundidos y condición de victoria — se ejecuta exclusivamente en el microprocesador RISC-V dentro de la FPGA**; la aplicación de PC es una terminal de entrada/salida remota, sin lógica de juego propia, de forma análoga al rol que cumplía la PC en el Proyecto 2. Como cada jugador cuenta con su propia pantalla (VGA para el Jugador 1, aplicación de PC para el Jugador

2), ninguno de los dos puede observar en ningún momento la disposición de barcos del otro antes de descubrirla mediante disparos.

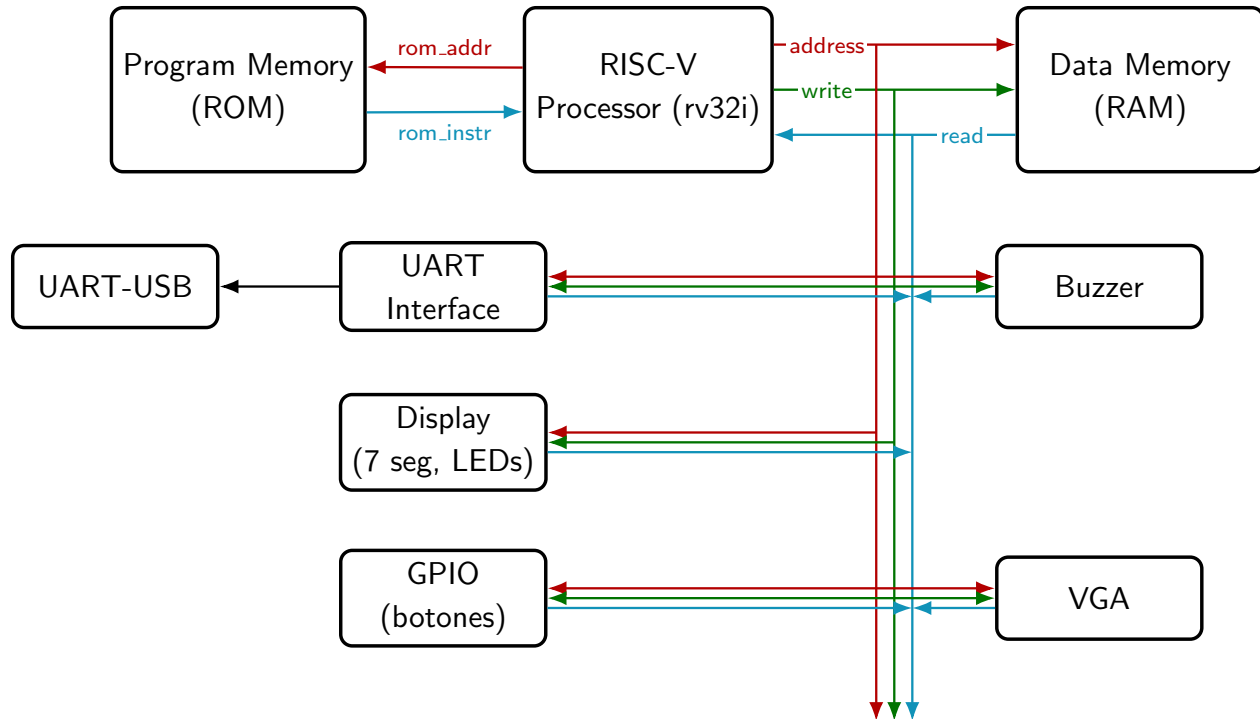


Figura 1: Arquitectura general del sistema: la memoria de programa (ROM) se conecta al microprocesador mediante los buses dedicados `rom_addr/rom_instr`, mientras que la memoria de datos (RAM) y todos los periféricos — UART hacia la aplicación de PC del Jugador 2, entradas del Jugador 1, VGA, *buzzer* y displays (7 segmentos, LED) — comparten un mismo bus de 3 líneas: `address`, `write` y `read`.

Para el desarrollo de este proyecto se deberán acatar las siguientes indicaciones:

1. Lea y trate de comprender todo el trabajo solicitado antes de iniciarlo.
2. Durante el desarrollo del diseño y la validación, asegúrese de documentar todas las tablas de verdad, circuitos, figuras, diagramas (flujo, estado, bloques, tiempos), entre otros, que sean necesarios. Esto será **fundamental** para la revisión del diseño y para la documentación del proyecto. Siga los lineamientos del diseño modular.
3. Para la presentación funcional, se le pedirá que muestre algunos de los diseños propuestos, ya sea en implementación o simulación. Dicha selección se hará el día de la presentación.
4. Todos los bloques de diseño deben ser **sintetizables** en SystemVerilog. El sistema debe funcionar con un único reloj de entrada de 100 MHz, a partir del cual se derivarán, mediante PLL, las frecuencias de reloj necesarias para el periférico VGA.

Los sistemas desarrollados en este proyecto deben ser completamente **sintetizables**, descritos utilizando diseño RTL con SystemVerilog. Tome en cuenta que estará desarrollando, como parte de su solución, lógica combinacional y secuencial, por lo que los bloques desarrollados deben

estar libres de elementos de memoria no intencionados, especialmente *latches*. Compruebe, por medio de los mensajes, visualización del *netlist* de la herramienta, y la utilización del *linter*, que no se generen dichos elementos como parte del proceso de síntesis. Las simulaciones deben ser presentadas a nivel de **post-implementación** temporizadas.

2. Objetivos

1. Diseñar e implementar un microprocesador de 32 bits, basado en el subconjunto de instrucciones *rv32i* de la arquitectura RISC-V, capaz de ejecutar un programa en ensamblador que controla completamente una aplicación interactiva.
2. Diseñar periféricos mapeados en memoria para generación de video VGA, lectura de entradas del jugador local e indicadores locales (displays de 7 segmentos, LED, *buzzer*), reutilizando además el periférico UART desarrollado en el Proyecto 2 para la comunicación con el jugador remoto.
3. Implementar en lenguaje ensamblador RISC-V la lógica completa de un juego de Batalla Naval para dos jugadores: colocación de barcos, alternancia de turnos, validación de disparos y determinación de la partida, coordinando un jugador local (FPGA/VGA) y un jugador remoto (aplicación de PC vía UART).
4. Diseñar la aplicación de PC (por ejemplo, en Python) que actúa como interfaz remota del Jugador 2, y el protocolo de comunicación sobre UART entre dicha aplicación y el microprocesador.
5. Aplicar metodologías de diseño modular y validación por etapas, separando el núcleo del procesador, las memorias y cada periférico como bloques independientes y verificables.
6. Fortalecer la capacidad de razonar sobre la relación entre hardware y software en un sistema embebido, comprendiendo cómo las decisiones de arquitectura del procesador y de la interfaz de periféricos condicionan el programa que se ejecuta sobre ellos.

3. Investigación previa

1. Investigue la arquitectura RISC-V, particularmente el subconjunto de instrucciones *rv32i*: formato de instrucciones, codificación y banco de registros.
2. Investigue organizaciones clásicas de *datapath* para procesadores de ciclo único y su relación con el diseño de una unidad de control.
3. Investigue el mapeo de periféricos en memoria (*memory-mapped I/O*) y buenas prácticas para el diseño de registros de control/estado/datos de 32 bits.
4. Investigue los fundamentos de la generación de video VGA: temporización de sincronismos horizontal y vertical, resolución 640×480 a 60 Hz, y el reloj de píxel de 25 MHz.

5. Investigue estrategias de generación de gráficos simples orientadas a *tiles* (bloques de color/carácter) como alternativa eficiente a un *framebuffer* completo, apropiada para procesadores con acceso limitado a memoria.
6. Investigue técnicas de *debouncing* para pulsadores y su importancia en interfaces de navegación mediante *push buttons*.
7. Investigue las reglas clásicas del juego de Batalla Naval (*Battleship*): colocación de barcos, mecánica de disparo y condición de victoria.
8. Repase el protocolo UART y el periférico UART desarrollado en el Proyecto 2, que será reutilizado en este proyecto para la comunicación con el jugador remoto.
9. Investigue el uso de la biblioteca `pyserial` (u otra equivalente) para el desarrollo de aplicaciones de PC que se comuniquen con un dispositivo serie.

4. Especificaciones

4.1. Resumen del sistema

El sistema debe ejecutar, sobre el microprocesador RISC-V diseñado por el equipo, un único programa en ensamblador que implementa el flujo completo del juego de Batalla Naval descrito en la Sección 4.2. **Todo el control del juego debe implementarse en software (ensamblador) ejecutado por el procesador**; los periféricos únicamente exponen recursos de entrada/salida (video, botones, displays, sonido, comunicación serial) y no deben contener lógica de reglas del juego. La única excepción es el manejo de bajo nivel estrictamente necesario para operar cada periférico (por ejemplo, generación de temporización VGA, *debouncing* de botones o formación de tramas UART a nivel de bit), el cual sí corresponde al hardware del periférico. De igual forma, la aplicación de PC del Jugador 2 se considera una terminal de entrada/salida remota y **no debe contener lógica de reglas del juego**.

4.2. Aplicación del juego: Batalla Naval

- El juego es para **dos jugadores**: el Jugador 1 utiliza la tarjeta de FPGA con despliegue en VGA y botones locales, y el Jugador 2 utiliza la aplicación de PC conectada por UART.
- Cada jugador tiene un tablero propio de **8×8 casillas**.
- Cada jugador coloca una flota de **3 barcos**, de tamaños 4, 3 y 2 casillas, en orientación horizontal o vertical, sin que los barcos se traslapen ni salgan del tablero.
- Una casilla puede encontrarse en uno de cuatro estados visibles: *agua* (sin disparar), *barco propio* (solo visible al dueño del tablero), *impacto* (disparo acertado) o *fallo* (disparo sin acertar).
- Un barco se considera **hundido** cuando todas sus casillas han sido impactadas.

- La partida se gana cuando un jugador hunde **todos** los barcos del tablero contrario.
- En ningún momento el Jugador 1 debe poder observar en VGA la disposición de barcos del Jugador 2, ni la aplicación de PC del Jugador 2 debe recibir por UART información sobre la disposición de barcos del Jugador 1 más allá de los resultados de sus propios disparos.

4.3. Fases de la partida

4.3.1. Fase de colocación de barcos

Dado que cada jugador cuenta con su propia pantalla privada (VGA para el Jugador 1, aplicación de PC para el Jugador 2), ambos pueden colocar su flota de forma independiente, sin necesidad de compartir ni traspasar un mismo dispositivo de despliegue.

1. Al iniciar una partida (o tras `BTN_RST`), el sistema muestra en VGA el tablero vacío del Jugador 1 e invita a colocar la flota; simultáneamente, envía por UART una notificación de inicio de colocación para que la aplicación de PC del Jugador 2 solicite la ubicación de su flota.
2. El Jugador 1 coloca sus 3 barcos sobre su propio tablero en VGA, usando los botones de navegación para mover el cursor, el botón de selección para rotar la orientación del barco, y `BTN_OK` para confirmar cada colocación. El sistema debe impedir colocaciones inválidas (traslape o fuera del tablero) y señalizarlo (por ejemplo, mediante el *buzzer*).
3. El Jugador 2 coloca sus 3 barcos mediante la aplicación de PC, la cual transmite por UART, para cada barco, su identificador, la casilla inicial y la orientación. El microprocesador valida cada colocación (traslape o fuera del tablero) y responde por UART si fue aceptada o rechazada; la aplicación de PC debe solicitar nuevamente la colocación en caso de rechazo.
4. El sistema debe permitir que ambos jugadores coloquen su flota de forma concurrente (el avance de uno no debe bloquear al otro), y debe llevar el control independiente de cuál de los dos ya completó su colocación.
5. Una vez que ambos jugadores han colocado sus 3 barcos, el sistema inicia la fase de batalla, notificando por UART a la aplicación de PC y actualizando el LED de estado y el despliegue en VGA.

4.3.2. Fase de batalla

1. El sistema indica mediante VGA, displays de 7 segmentos y/o LED cuál jugador tiene el turno activo, y notifica el cambio de turno a la aplicación de PC por UART.
2. Si el turno activo es del **Jugador 1**, este mueve el cursor sobre el tablero **del oponente** en VGA (mostrando únicamente casillas de agua, impacto o fallo; nunca la posición de barcos no descubiertos) y confirma un disparo con `BTN_OK`. Si el turno activo es del **Jugador 2**, la aplicación de PC solicita al usuario una casilla del tablero rival y la transmite por UART.

3. El sistema valida la casilla seleccionada: si ya fue disparada previamente, la entrada debe **ignorarse** (no consume turno). En caso contrario, se determina si hay impacto o fallo, se actualiza en RAM el tablero correspondiente, se refleja el resultado en VGA (si el disparo fue sobre el tablero del Jugador 2) y se notifica por UART a la aplicación de PC el resultado del disparo (propio, si fue el Jugador 2 quien disparó, o recibido, si fue el Jugador 1 quien disparó sobre el tablero del Jugador 2).
4. Cada impacto o fallo debe producir una retroalimentación sonora distinta mediante el *buzzer*. Al hundirse un barco completo, se debe generar una tercera señal sonora distintiva.
5. Tras cada disparo válido, el turno pasa al otro jugador.
6. El ciclo se repite hasta que todos los barcos de un jugador han sido hundidos.

4.3.3. Fin de la partida

Al hundirse todos los barcos de un jugador, el sistema debe:

- desplegar en VGA el resultado de la partida indicando el jugador ganador,
- notificar por UART a la aplicación de PC el resultado de la partida y un resumen (por ejemplo: jugador ganador, número de disparos totales y número de barcos hundidos por cada jugador),
- generar una secuencia sonora distintiva de victoria mediante el *buzzer*,
- actualizar en los displays de 7 segmentos el contador acumulado de partidas ganadas por cada jugador (00–99) desde el último reinicio general del sistema,
- permanecer en la pantalla de resultado hasta que se presione BTN_RST, momento en el cual el sistema regresa al inicio de una nueva fase de colocación, conservando el contador acumulado de partidas ganadas.

4.4. Microprocesador y plataforma computacional

4.4.1. Núcleo RISC-V

Los estudiantes deberán diseñar e implementar un microprocesador de 32 bits. Dicho microprocesador debe implementar las instrucciones necesarias del subconjunto rv32i para la ejecución del programa. Considere la siguiente lista como una base de las instrucciones que el procesador debe soportar. Debe asegurarse de que el microprocesador sea **sintetizable**, de otra manera no se podrá ejecutar en la tarjeta con FPGA.

lw, sw,
sll, slli, srl, srli, sra, srai,
add, and, xor, or, sub,

```

addi, andi, xori, ori,
beq, bne, blt, bge,
slt, slti, sltu, sltui,
jal, jalr

```

El microprocesador utilizará una memoria ROM para almacenar el programa y una memoria RAM para almacenar los datos (tableros, turno activo, contadores de partidas, etc.). Dichas memorias se accederán mediante buses **independientes**: ProgAddress_o[31:0] y ProgIn_i[31:0] para instrucciones, y DataAddress_o[31:0], DataOut_o[31:0] y DataIn_i[31:0] para datos, como se muestra en la Figura 2.

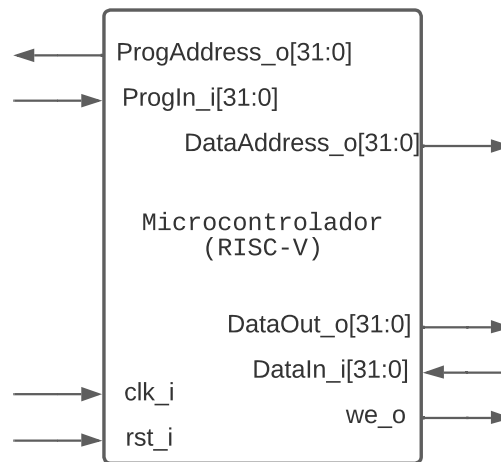


Figura 2: Diagrama de alto nivel del *core* para el microprocesador.

El sistema recibe como entrada principal el reloj de 100 MHz. Los estudiantes deberán generar, empleando un PLL, la señal de reloj de píxel de 25 MHz requerida por el periférico VGA, así como cualquier otra frecuencia de reloj necesaria para el correcto funcionamiento del periférico UART reutilizado del Proyecto 2.

4.4.2. Mapa de memoria

El mapa de memoria del sistema se organiza en tres regiones: ROM, para el programa (0x0000_0000 a 0x0000_1FFF), RAM, para los datos (0x0000_2000 a 0x0000_2FFF), y espacio de periféricos (0x0001_0000 a 0x0001_FFFF). La ejecución del programa inicia en 0x0000_0000 (vector de reset).

4.4.3. Direcciones de periféricos

La siguiente tabla presenta el mapa de memoria propuesto para los periféricos del microprocesador a desarrollar.

Periférico	Registro	Offset	Dirección
UART	Control/Estado	0x00	0x0001_0040
	Datos TX	0x04	0x0001_0044
	Datos RX	0x08	0x0001_0048
Entradas Jugador 1	Estado	0x00	0x0001_0120
Displays 7 segmentos	Datos	0x00	0x0001_0130
LED de estado	Datos	0x00	0x0001_0138
Buzzer	Control	0x00	0x0001_0140

A diferencia de los periféricos anteriores, la memoria de video del periférico VGA **no** se organiza como un conjunto reducido de registros, sino como un rango de memoria dedicado que el CPU accede directamente con instrucciones *lw/sw*, tal como accede a la RAM de datos:

Región	Rango de direcciones	Contenido
Memoria de video (VGA)	0x0001_1000 – 0x0001_17FF	mapa de <i>tiles</i> (una palabra de 32 bits por casilla)

4.5. Periféricos

4.5.1. Periférico VGA (diseño del equipo)

Cada grupo debe diseñar su propio periférico de video, responsable de generar la señal VGA a 640×480 píxeles a 60 Hz. Dado que el microprocesador no tiene capacidad práctica de escribir un *framebuffer* completo píxel por píxel, el periférico debe organizarse mediante un enfoque de **mapa de bloques (tiles)**: la pantalla se divide en una cuadrícula de bloques de color sólido (por ejemplo, 20×15 bloques de 32×32 píxeles), suficiente para representar ambos tableros de 8×8 y la información de HUD (turno activo, partidas ganadas, mensajes de traspaso).

A diferencia de los demás periféricos del proyecto, el VGA **no** se controla mediante registros de comando/estado: se expone directamente como una **memoria de video** mapeada en el rango 0x0001_1000–0x0001_17FF, con una palabra de 32 bits por casilla de la cuadrícula. El CPU actualiza una casilla con una única instrucción *sw*, sin necesidad de bits de start ni de esperar busy/done:

$$\text{dirección} = \text{VGA_BASE} + (\text{fila} * \text{NUM_COLUMNAS} + \text{columna}) * 4$$

Definición mínima del contenido de cada palabra:

- bits [2:0]: color de la casilla (mínimo 4 valores distintos y distinguibles: agua, barco propio, impacto, fallo; se sugiere reservar codificaciones adicionales para HUD).
- bits [7:3]: reservado/uso del equipo, por ejemplo para códigos de carácter/símbolo si el equipo implementa despliegue de texto en el HUD.
- bits [31:8]: reservado/uso del equipo (documentar).

Internamente, el periférico se implementa como una memoria de **doble puerto**: un puerto síncrono al reloj del sistema, accedido por el CPU mediante `write_enable_i/addr_i/wdata_i` igual que la RAM de datos, y un segundo puerto de solo lectura, sincronizado al reloj de píxel, que la lógica de generación de video lee de forma continua para producir la señal VGA. El equipo debe documentar el tratamiento de sincronización entre ambos dominios de reloj. No es necesario un bit de `clear` en hardware: limpiar la pantalla es una operación de software, recorriendo con un lazo las posiciones de la memoria de video y escribiendo en cada una el color de fondo.

Requisitos funcionales del periférico VGA:

- Generación estable de los sincronismos horizontal y vertical según el estándar 640×480 a 60 Hz.
- Representación simultánea, sin parpadeo perceptible, de la cuadrícula completa del tablero en despliegue y del HUD.
- Actualización de una casilla en un único ciclo de escritura del CPU, sin bloquear la ejecución del programa.
- El equipo debe documentar y justificar la resolución exacta de la cuadrícula de bloques elegida y su relación con el tamaño de los tableros y la información de HUD requerida.

4.5.2. Periférico de entradas del Jugador 1

Cada grupo debe diseñar un periférico de lectura de entradas (botones/*switches*) con *debouncing* adecuado, mapeado en un único registro de **ESTADO** (offset 0x00). Como mínimo debe exponer, en bits independientes de dicho registro, el estado ya filtrado (sin rebotes) de: navegación (arriba, abajo, izquierda, derecha), selección/rotación (BTN_SEL), confirmación (BTN_OK) y reinicio (BTN_RST). Estas entradas corresponden exclusivamente al Jugador 1; el Jugador 2 interactúa con el sistema únicamente por medio del periférico UART. El equipo debe documentar el mapeo exacto de bits utilizado.

4.5.3. Periférico UART (reutilizado)

Este proyecto reutiliza el periférico UART desarrollado en el Proyecto 2, incluyendo su interfaz de registros (Control, Datos TX, Datos RX) y su operación a 115200 baudios. A diferencia del Proyecto 2, donde la PC era una terminal de entrada/salida simple, en este proyecto el periférico UART constituye el **único canal de interacción del Jugador 2** con la partida: por él se transmiten tanto las órdenes del jugador remoto (colocación de barcos, disparos) como toda la información que la aplicación de PC necesita para desplegar el estado de la partida.

Protocolo de aplicación sobre UART. Sobre el enlace UART, además de la interfaz de registros del periférico, cada equipo debe definir y documentar un protocolo de aplicación mínimo con los siguientes mensajes:

- PC → FPGA:

- *Colocación de barco*: identificador del barco (0–2), casilla inicial (fila, columna) y orientación (horizontal/vertical).
 - *Disparo*: casilla objetivo (fila, columna) sobre el tablero del Jugador 1.
- **FPGA → PC**: al menos los siguientes eventos deben notificarse a la PC:
 - aceptación o rechazo de cada colocación de barco recibida, con motivo del rechazo (traslape o fuera de tablero);
 - inicio de la fase de batalla, una vez ambos jugadores completaron su flota;
 - cambio de turno (indicando a quién corresponde);
 - resultado de cada disparo propio del Jugador 2 (impacto/fallo/barco hundido) sobre el tablero del Jugador 1;
 - notificación de cada disparo recibido del Jugador 1 sobre el tablero del Jugador 2, con su resultado, de forma que la aplicación de PC pueda mantener actualizada la vista del tablero propio;
 - resultado final de la partida (jugador ganador y resumen de la partida).

El formato exacto de cada trama (delimitadores, longitud, orden de campos) queda a criterio del equipo, siempre que sea consistente y esté completamente documentado. Cualquier byte recibido que no corresponda a un mensaje válido debe ser descartado por la FPGA sin afectar la partida en curso; la aplicación de PC debe además validar la entrada del usuario antes de transmitirla.

4.5.4. Indicadores y controles locales

- **Displays de 7 segmentos**: utilice al menos 4 dígitos. Se sugiere asignar 2 dígitos al contador acumulado de partidas ganadas por el Jugador 1 y 2 dígitos al contador acumulado de partidas ganadas por el Jugador 2 (00–99 cada uno) desde el último reinicio general del sistema.
- **LED de estado**: indique con al menos un LED si el sistema se encuentra en la fase de colocación de barcos, en fase de batalla, o mostrando el resultado final, de forma claramente distinguible.
- **Buzzer**: genere una retroalimentación sonora distinta para: disparo con impacto, disparo con fallo, barco hundido, colocación inválida de barco, y fin de partida (victoria).
- **Botón central** (BTN_RST): reinicia la partida en curso en cualquier momento, regresando al inicio de una nueva fase de colocación de barcos y conservando el contador acumulado de partidas ganadas.

4.5.5. Interfaz estándar de periféricos

Para los periféricos de registros (UART, entradas del Jugador 1, displays, LED y buzzer) se debe usar una interfaz digital común de 32 bits para facilitar integración y reutilización, con

`addr_i[1:0]` seleccionando uno de hasta cuatro registros internos. El periférico VGA es la excepción: al comportarse como una memoria, requiere un campo de dirección más ancho (suficiente para indexar las 20×15 casillas del mapa de *tiles*) en lugar de `addr_i[1:0]`, pero conserva las mismas señales `clk_i`, `rst_i`, `write_enable_i`, `wdata_i[31:0]` y `rdata_o[31:0]`. La interfaz mínima para los periféricos de registros se define así:

Señal	Dirección	Descripción
<code>clk_i</code>	Entrada	Reloj del sistema.
<code>rst_i</code>	Entrada	Reinicio síncrono o asíncrono, según defina el equipo y documento.
<code>write_enable_i</code>	Entrada	Indica escritura cuando está en 1.
<code>addr_i[1:0]</code>	Entrada	Dirección de registro interno (00, 01, 10, 11).
<code>wdata_i[31:0]</code>	Entrada	Dato de escritura hacia el periférico.
<code>rdata_o[31:0]</code>	Salida	Dato leído desde el periférico en la dirección seleccionada por <code>addr_i[1:0]</code> .

Notas de uso:

- Cuando `write_enable_i=1`, el periférico escribe `wdata_i[31:0]` en el registro seleccionado por `addr_i[1:0]`.
- Cuando `write_enable_i=0`, la lectura se realiza por `rdata_o[31:0]` desde el registro seleccionado por `addr_i[1:0]`.
- Los registros deben mapearse con `addr_i[1:0]` y documentarse claramente.

4.6. Aplicación de PC en Python

Como parte de la especificación del sistema, se debe implementar una aplicación en Python (u otro lenguaje equivalente acordado con el profesor) para la PC, que actúe como interfaz del Jugador 2 remoto mediante UART. Esta aplicación debe:

- solicitar al Jugador 2 la colocación de cada barco (casilla inicial y orientación) y transmitirla a la FPGA, reintentando la solicitud si la colocación es rechazada,
- mostrar en todo momento el tablero propio del Jugador 2 (incluyendo los barcos y los disparos recibidos del Jugador 1) y el estado conocido del tablero rival (únicamente impactos y fallos de los disparos propios),
- indicar claramente de quién es el turno activo,
- solicitar al Jugador 2 la casilla a disparar cuando le corresponda el turno, y transmitirla a la FPGA,
- mostrar el resultado de cada disparo (propio o recibido) y el resultado final de la partida cuando la FPGA lo notifique,
- manejar y reportar entradas inválidas (por ejemplo, coordenadas fuera de rango) sin afectar la estabilidad de la partida ni bloquear la aplicación.

4.7. Flujo detallado de la aplicación en ensamblador

El programa en ensamblador debe implementarse como un único flujo determinístico. En la etapa de **inicialización**, el CPU configura los periféricos (VGA, entradas, UART, displays, LED, buzzer), limpia la memoria de video y las estructuras de datos de ambos tableros en RAM, e inicializa los contadores de partidas ganadas.

A partir de allí, el programa recorre las fases descritas en la Sección 4.2: colocación de barcos de ambos jugadores (Jugador 1 por VGA/botones, Jugador 2 por UART), fase de batalla con lectura de entradas locales y de tramas UART, validación de disparos, actualización de VGA/buzzer/displays y notificación por UART, y finalmente la fase de fin de partida. El equipo debe documentar la organización de datos en RAM elegida para representar cada tablero (por ejemplo, un arreglo de 64 posiciones por jugador con el estado de cada casilla) y la estructura del programa (subrutinas, convenciones de llamado, manejo de la pila si aplica).

4.8. Arquitectura del sistema

La arquitectura interna del sistema queda **abierta** para que cada equipo proponga su diseño, tanto a nivel del *datapath* y unidad de control del procesador como de la organización interna de cada periférico, siempre que se cumplan todas las especificaciones funcionales, de interfaz y de validación establecidas en este documento.

5. Entregables

1. Planteamiento del diseño:

- propuesta de arquitectura modular con enfoque **top-down**, siguiendo los principios de diseño modular, para el procesador y cada periférico;
- objetivo de cada diagrama/módulo y rol dentro del sistema,
- definición de entradas, salidas y señales internas relevantes por bloque principal,
- entrega del planteamiento en docs/planteamiento.md (o archivo equivalente dentro de docs/).

2. Funcionalidad final:

- demostración de una partida completa de Batalla Naval entre dos jugadores, incluyendo colocación de barcos, fase de batalla y determinación del ganador,
- evidencia de que ningún jugador puede observar la disposición de barcos del otro antes de descubrirla mediante disparos,
- evidencia de comunicación bidireccional confiable entre la FPGA y la aplicación de PC del Jugador 2,
- evidencia de visualización correcta en VGA del tablero del Jugador 1 y del HUD,
- evidencia de visualización correcta en la aplicación de PC del tablero del Jugador 2 y del estado del tablero rival,

- evidencia de retroalimentación sonora (buzzer) para impacto, fallo, barco hundido, colocación inválida y victoria,
- evidencia de actualización de displays de 7 segmentos y LED de estado,
- simulación **post-implementación** temporizada del sistema, cubriendo al menos la ejecución de un fragmento representativo del programa y la validación de un disparo,
- evidencia de **tests de autochequeo** asociados a la funcionalidad principal del procesador y de los periféricos.

3. Documentación técnica en formato Markdown dentro de docs/ (por ejemplo, docs/informe.md):

- como referencia de formato para la documentación técnica, puede consultar este ejemplo,
- aspectos teóricos relevantes para sustentar el diseño propuesto,
- enfoque de la solución (decisiones de diseño y justificación técnica) para el procesador y cada periférico,
- descripción formal de interfaces de módulos,
- definición de la memoria de video del periférico VGA, de los registros del periférico de entradas del Jugador 1 y del protocolo de aplicación sobre UART (organización de memoria/registros, bits, tramas, semántica),
- diagramas de estado del control principal del juego y de los periféricos relevantes,
- estrategia de validación (simulación y pruebas en FPGA),
- evidencia de simulaciones con pruebas de autochequeo (criterios de pase/fallo y resultados),
- resultados y evidencias funcionales (capturas, tablas y observaciones), incluyendo uso de recursos y análisis de *timing*,
- breve análisis crítico de lo realizado (logros, limitaciones y mejoras posibles),
- análisis de problemas encontrados y cómo se resolvieron.

4. Repositorio y trazabilidad:

- código fuente HDL y *constraints*,
- programa en ensamblador RISC-V del juego,
- aplicación de PC del Jugador 2,
- bancos de prueba y **tests de autochequeo**,
- scripts de simulación/síntesis (si aplica),
- README.md principal con explicación clara de la estructura del proyecto/repositorio,
- historial de trabajo con *commits* suficientes y descriptivos,
- uso de flujo de desarrollo moderno con **issue tracking** (incluyendo asignación de tareas por integrante), **feature branches**, **commits significativos** y uso de **merge/pull requests** entre integrantes del equipo, siguiendo *git workflow*.

6. Evaluación

Este proyecto tiene una duración de tres semanas a partir de su inicio. El planteamiento del diseño debe ser validado con el profesor **previo a la presentación funcional**. Ningún proyecto será evaluado sin el visto bueno desde la perspectiva de diseño.

Las fechas oficiales de este proyecto son:

- **Inicio:** G1: 17.09.2026 / G2: 17.09.2026
- **Planteamiento del diseño:** G1: 24.09.2026 / G2: 23.09.2026
- **Entrega del informe técnico:** G1: 08.10.2026 / G2: 07.10.2026
- **Fin:** G1: 08.10.2026 / G2: 07.10.2026

El presente proyecto tiene un valor del **15 %** de la nota final del curso y se evaluará mediante cuatro rubros, cada uno calificado con una rúbrica común a todos los proyectos del curso. El detalle de cada rúbrica se presenta en el Anexo A.

1. Diseño	20 %
2. Presentación funcional	50 %
3. Informe técnico	15 %
4. Repositorio	15 %

El formato de entrega de cada documento (Markdown enriquecido, ubicado en la carpeta correspondiente del repositorio) se detalla junto a su rúbrica en el Anexo A.

A. Anexo: Rúbricas de evaluación

Las siguientes rúbricas son comunes a todos los proyectos del curso y se detallan a continuación para que el estudiante cuente con los criterios de evaluación completos dentro de este mismo documento.

A.1. Diseño

Documentación del diseño

Rúbrica aplicable al planteamiento del diseño entregado en cada proyecto del curso. El diseño debe seguir la metodología de diseño modular vista en clase. Evalúa la calidad de los diagramas, la justificación técnica de las decisiones tomadas, la completitud del documento y la claridad general de la presentación.

Formato de entrega: el planteamiento del diseño debe entregarse como un archivo Markdown (.md) enriquecido, ubicado en la carpeta docs/diseño/ del repositorio del proyecto. No se aceptan formatos alternativos, como un PDF exportado desde un procesador de texto externo, como entrega oficial.

Nivel	Descripción del criterio
Diagramas y representaciones del diseño (peso: 30 %)	
Excelente	Diagramas claros, correctos y bien organizados (bloques, flujo, estados, esquemas u otros). Reflejan correctamente la arquitectura del sistema y facilitan la comprensión.
Satisfactorio	Diagramas mayormente correctos, pero con detalles poco claros, incompletos o escasa explicación.
Regular	Diagramas simplificados o con errores que dificultan entender el diseño.
Deficiente	No hay diagramas o son incorrectos/confusos.
Justificación y decisiones de diseño (peso: 30 %)	
Excelente	Explica y justifica claramente las decisiones de diseño (arquitectura, señales, ecuaciones, implementación, etc.). Demuestra razonamiento técnico sólido.
Satisfactorio	Presenta las decisiones principales, pero con justificación limitada.
Regular	Justificación mínima o superficial de las decisiones de diseño.
Deficiente	No se explican ni justifican las decisiones de diseño.
Completitud del documento de diseño (peso: 20 %)	
Excelente	Incluye todos los elementos solicitados (objetivos, tablas, ecuaciones, diagramas, estrategia de implementación, plan de pruebas, etc.) de forma completa y bien estructurada.

Nivel	Descripción del criterio
Satisfactorio	Incluye la mayoría de los elementos requeridos, con pequeños faltantes o secciones poco desarrolladas.
Regular	Varias secciones importantes faltan o están incompletas.
Deficiente	El documento es muy incompleto o no sigue la estructura solicitada.
Claridad y calidad de la documentación (peso: 20 %)	
Excelente	Documento bien organizado, claro y profesional. Uso adecuado de Markdown, tablas, figuras y notación técnica.
Satisfactorio	Documento comprensible pero con problemas menores de organización o presentación.
Regular	Organización deficiente o redacción poco clara.
Deficiente	Documento difícil de leer o mal presentado.

A.2. Presentación funcional

Presentación funcional

Rúbrica aplicable a la demostración funcional del Proyecto 3. Evalúa el funcionamiento del microprocesador RISC-V y su programa en ensamblador, la comunicación con la aplicación de PC del Jugador 2 por UART, la operación de los periféricos locales del Jugador 1 (VGA, entradas, indicadores), la correcta implementación de las reglas del juego de Batalla Naval, la calidad de la verificación por simulación, y la defensa técnica del diseño ante preguntas del profesor.

Nivel	Descripción del criterio
Microprocesador RISC-V (peso: 15 %)	
Excelente	El microprocesador ejecuta de forma correcta y estable el subconjunto de instrucciones rv32i requerido, con acceso independiente y sin errores a las memorias de programa y datos, y sin bloqueos durante la ejecución completa del juego.
Satisfactorio	El microprocesador ejecuta el programa correctamente en la mayoría de los casos, con fallas ocasionales asociadas a alguna instrucción o a la interacción con memoria/periféricos.
Regular	El microprocesador presenta fallas frecuentes en la ejecución de instrucciones o en el acceso a memoria, aunque se logra demostrar parcialmente su funcionamiento.
Deficiente	El microprocesador no ejecuta el programa de forma reconocible, o el sistema se bloquea antes de completar una partida.
Comunicación UART y aplicación de PC (Jugador 2) (peso: 15 %)	

Nivel	Descripción del criterio
Excelente	La comunicación entre la FPGA y la aplicación de PC funciona de forma confiable y sin pérdida de datos; la aplicación de PC permite colocar barcos y disparar correctamente, y mantiene actualizado en todo momento el tablero propio y el estado conocido del tablero rival del Jugador 2.
Satisfactorio	La comunicación y la aplicación de PC funcionan correctamente en la mayoría de los casos, con errores ocasionales de sincronización o de actualización de la vista del Jugador 2 que no impiden continuar la partida.
Regular	La comunicación o la aplicación de PC presentan errores frecuentes que afectan visiblemente el desarrollo de la partida, aunque se logra demostrar que existe transmisión de datos entre ambos.
Deficiente	No existe comunicación funcional entre la FPGA y la aplicación de PC, o el Jugador 2 no puede colocar barcos ni disparar a través de ella.

Periférico VGA e interfaz local del Jugador 1 (peso: 15 %)	
Excelente	El periférico VGA genera una imagen estable, sin parpadeo ni corrimiento, mostrando correctamente el tablero propio y el tablero rival del Jugador 1 y la información de HUD; los botones de navegación, selección y confirmación funcionan libres de rebotes.
Satisfactorio	La visualización VGA y las entradas del Jugador 1 funcionan correctamente en la mayoría de los casos, con fallas ocasionales y menores en la actualización de casillas, del HUD o en el manejo de botones.
Regular	La visualización o las entradas presentan fallas evidentes — imagen inestable, casillas mal ubicadas, rebotes no filtrados — que afectan la jugabilidad, aunque se logra demostrar que el periférico opera.
Deficiente	El periférico VGA no genera una imagen reconocible, o las entradas del Jugador 1 no responden de forma reconocible.

Lógica del juego (Batalla Naval) en ensamblador (peso: 15 %)	
Excelente	El programa en ensamblador implementa correctamente la colocación de barcos de ambos jugadores, la alternancia de turnos, la validación de disparos, la detección de barcos hundidos y la condición de victoria, coordinando correctamente al jugador local y al remoto.
Satisfactorio	La lógica del juego funciona correctamente en la mayoría de los casos, con desviaciones menores respecto a las reglas especificadas (por ejemplo, en la detección de barco hundido o en la coordinación de turnos entre ambos jugadores).
Regular	La lógica del juego presenta fallas evidentes en el manejo de turnos, colocación de barcos o determinación de impactos, aunque la partida es parcialmente jugable.
Deficiente	La lógica del juego no funciona de forma reconocible, o la partida no sigue las reglas especificadas de Batalla Naval.

Nivel	Descripción del criterio
Simulación autoverificable (RTL/Verilog) (peso: 15 %)	
Excelente	Presenta simulaciones autoverificables (<i>testbenches</i> con verificación automática de resultados) para los bloques principales del sistema (núcleo RISC-V y periféricos), con reporte claro de aciertos/fallos y cobertura de los casos relevantes, incluyendo condiciones de borde.
Satisfactorio	Presenta simulaciones autoverificables para la mayoría de los bloques, aunque con cobertura limitada de casos de borde o verificación parcialmente manual.
Regular	Las simulaciones presentadas requieren inspección manual de formas de onda para determinar su correctud; cobertura escasa de los bloques del diseño.
Deficiente	No presenta simulaciones autoverificables, o las presentadas no permiten verificar el comportamiento correcto del diseño.
Defensa y comprobación de dominio técnico (peso: 25 %)	
Excelente	Responde con precisión y seguridad a las preguntas técnicas sobre el diseño del microprocesador, los periféricos, el protocolo UART y el programa en ensamblador, justificando decisiones de arquitectura e implementación ante modificaciones o pruebas solicitadas por el profesor durante la presentación.
Satisfactorio	Responde correctamente a la mayoría de las preguntas técnicas, aunque con dudas puntuales o explicaciones incompletas sobre algunos bloques del diseño.
Regular	Responde de forma superficial o memorística, con dificultad para justificar decisiones de diseño o para explicar el funcionamiento ante cambios solicitados.
Deficiente	No logra explicar ni justificar el funcionamiento del diseño, o las respuestas evidencian que no comprende su propia implementación.

A.3. Informe técnico

Documentación técnica (informe)

Rúbrica aplicable al informe técnico final entregado en cada proyecto del curso. Evalúa la fundamentación teórica, la presentación y el análisis de los resultados, las conclusiones obtenidas y la calidad general del documento.

Formato de entrega: el informe técnico debe entregarse como un archivo Markdown (.md) enriquecido, ubicado en la carpeta docs/informe/ del repositorio del proyecto. No se aceptan formatos alternativos, como un PDF exportado desde un procesador de texto externo, como entrega oficial.

Nivel	Descripción del criterio
Fundamentación teórica (peso: 20 %)	
Excelente	Presenta los conceptos teóricos relevantes con claridad y correcta relación con el laboratorio/proyecto. Integra adecuadamente el marco teórico con el trabajo realizado.
Satisfactorio	Presenta la mayoría de los conceptos necesarios, aunque con poca profundidad o conexión limitada con el laboratorio/proyecto.
Regular	Marco teórico superficial o incompleto.
Deficiente	No presenta fundamentación teórica o es incorrecta.
Presentación de resultados (peso: 30 %)	
Excelente	Resultados completos y organizados (tablas, gráficas, simulaciones, mediciones). Claramente etiquetados y bien documentados.
Satisfactorio	Resultados mayormente completos, aunque con organización o explicación limitada.
Regular	Resultados incompletos o presentados de forma poco clara.
Deficiente	No presenta resultados relevantes o son incorrectos.
Análisis e interpretación de resultados (peso: 25 %)	
Excelente	Analiza los resultados de forma crítica, compara valores teóricos, simulados y experimentales cuando aplica, e identifica causas de diferencias o errores.
Satisfactorio	Presenta análisis general de los resultados, pero con poca profundidad o discusión limitada.
Regular	Análisis superficial, descriptivo o con escasa interpretación técnica.
Deficiente	No analiza los resultados o las conclusiones no se basan en los datos.
Conclusiones y aprendizaje obtenido (peso: 15 %)	
Excelente	Conclusiones claras, bien fundamentadas y directamente relacionadas con los resultados y objetivos del laboratorio/proyecto. Incluye reflexiones o lecciones aprendidas.
Satisfactorio	Conclusiones correctas pero generales o poco conectadas con los resultados.
Regular	Conclusiones superficiales o parcialmente relacionadas con el trabajo realizado.
Deficiente	No presenta conclusiones o son incorrectas.
Calidad y organización del documento (peso: 10 %)	
Excelente	Documento claro, bien estructurado y profesional. Uso adecuado de Markdown, figuras, tablas, ecuaciones y referencias.
Satisfactorio	Organización adecuada con pequeños problemas de claridad o formato.
Regular	Documento desorganizado o difícil de seguir.
Deficiente	Documento mal estructurado o incompleto.

A.4. Repositorio

Gestión de repositorio

Rúbrica aplicable al repositorio Git de cada proyecto del curso. Evalúa el historial de commits, el uso de ramas de desarrollo, la gestión de issues, la revisión colaborativa de cambios y la estructura del README.

Nivel	Descripción del criterio
Historial de commits y calidad de código (peso: 20 %)	
Excelente	Commits frecuentes, claros y descriptivos, reflejan avances reales del proyecto y buena trazabilidad. Código documentado y organizado correctamente.
Satisfactorio	Commits suficientes pero con mensajes genéricos o frecuencia irregular. Código sin comentar o desordenado.
Deficiente	Pocos commits, sin descripción clara o sin reflejar el desarrollo. No tiene código real.
Uso de ramas de desarrollo (peso: 20 %)	
Excelente	Uso sistemático de ramas por funcionalidad o integrante, siguiendo un flujo profesional (Git Flow o similar).
Satisfactorio	Se usan ramas ocasionalmente o sin convención clara.
Deficiente	Todo el trabajo se hace en una sola rama sin control de versiones adecuado.
Gestión de issues y seguimiento (peso: 20 %)	
Excelente	Issues bien definidos y organizados, usados para planificar tareas, reportar bugs y documentar avances.
Satisfactorio	Uso limitado de issues o sin seguimiento completo.
Deficiente	No se usan issues o no hay evidencia de planeamiento.
Revisión colaborativa (pull/merge requests) (peso: 20 %)	
Excelente	Uso activo de pull o merge requests con revisiones cruzadas entre miembros antes de integrar cambios.
Satisfactorio	Uso ocasional o sin revisión efectiva.
Deficiente	No se utilizan revisiones ni solicitudes de integración.
Estructura del README (peso: 20 %)	
Excelente	README completo y organizado: incluye propósito, dependencias, instalación, compilación y ejecución.
Satisfactorio	README con información parcial o poco detallada.
Deficiente	README ausente o sin instrucciones útiles.